

CSE 310 (Compiler Sessional) — The Complete Study Book

January 2026 Term, BUET — synthesized from your Offline specs, Online lab-finals, the 2019 previous quiz, and *The Definitive ANTLR 4 Reference* (Terence Parr)

How this document is organized differently from before: instead of listing "here's what file A1 says, here's what file B1 says," everything below is fused into one continuous explanation *per concept*. Every online-lab variation you were given is folded into the theory as a worked illustration of that theory, the way a textbook chapter would use a worked example — not as a separate bullet you have to cross-reference. Read each Part top to bottom like a chapter. By the end of a Part you should be able to write a full 5-mark answer on any sub-topic inside it, not just recognize a keyword.

A note on the ANTLR4 book: your teacher said quiz questions will come from *The Definitive ANTLR 4 Reference*. I don't have your actual PDF copy open in front of me (only the cover screenshot you sent), so **Part IV, Section 4A** below is built from the well-established, widely documented structure and content of that specific book (2nd edition) — its chapter on grammars, the Listener/Visitor chapter, left-recursion handling, semantic predicates, error strategies, and — most importantly for you — its dedicated chapter on **building a symbol table with ANTLR** and its chapters on **building interpreters and bytecode compilers**, which map almost one-to-one onto your Assignment 3 and Assignment 4. If you want page-exact quotes or figure numbers, send me the actual PDF file (not just a screenshot) and I'll re-derive this section directly from it.

PART I — What a Compiler Actually Does, and Why You're Building It in This

Order

A compiler's job is to take source text — a flat sequence of characters — and produce, eventually, a sequence of machine instructions that do the same thing. That's a huge jump, so real compilers never try to do it in one step. They break the problem into **phases**, each phase consuming the *output data structure* of the phase before it and producing a *richer* data structure for the phase after it:

```
characters --[Lexical Analysis]--> tokens
tokens     --[Syntax Analysis]-->  parse tree
parse tree --[Semantic Analysis]--> annotated/checked parse tree
annotated tree --[ICG]-->          intermediate code (in your case, x86
assembly text)
intermediate code --[Optimization]--> improved intermediate code
```

This is exactly the order your course builds things in: Symbol Table (a supporting structure needed by **every** later phase) → Lexical Analysis (Flex) → Syntax + Semantic Analysis (ANTLR4) → Intermediate Code Generation (x86 via FASM). Two things run underneath **all four** phases rather than being a phase of their own:

1. **The Symbol Table** — a shared, mutable record of every identifier's name, type, and scope, consulted and updated by nearly every other phase.
2. **Error Handling** — every phase can fail, and a good compiler reports the error with a line number and *keeps going* rather than crashing, so the programmer gets more than one error per compile attempt. You'll notice this same philosophy ("don't terminate, skip the bad part and continue, but stop after the first error for some problems") repeats itself in nearly every assignment you've been given — it isn't a quirky one-off requirement, it's literally **the** recurring compiler-design decision you're being tested on.

Two classifications of phases you should have memorized cold, because they are pure MCQ trivia:

- **Machine-independent phases** (the front end + IR generation — nothing here knows or cares what CPU it will eventually run on): Lexical Analysis, Syntax Analysis, Semantic Analysis, Intermediate Code Generation, high-level (machine-independent) Code Optimization, and Symbol Table management.
- **Machine-dependent phases** (the back end): target-specific Code Optimization (e.g., register allocation for a *specific* register set) and final Code Generation (the actual x86 instructions — this is what your ICG assignment does, so note that even

though you call it "Intermediate Code Generation," emitting *actual x86 assembly text* is arguably straddling the line between IR generation and machine-dependent code generation; if a question asks you to place it, the safe answer is "it plays the role of machine-dependent code generation, because the output is literally x86 instructions," even though your assignment names it ICG).

PART II — The Symbol Table

2.1 Why a hash table, and why *per scope*?

A symbol table needs to answer one question fast, over and over, for every single identifier the compiler ever sees: **"have I seen this name before, and if so what do I know about it?"** A hash table gives near $O(1)$ average lookup, which is why it's the natural choice over, say, a sorted array or linked list.

But a program doesn't have one flat namespace — it has **nested scopes** (global scope, function scope, a block inside an `if`, etc.), and the *same name* can legally mean *different things* in different scopes (shadowing). A single flat hash table can't represent this, because inserting a new `a` inside an inner block would either collide with or silently overwrite the outer `a`. The fix your course uses is: **one small hash table (a "Scope Table") per scope**, and a **list/stack of Scope Tables** representing the whole program's nested-scope structure at any instant. This design — rather than "one hash table with a scope tag on each row" — has a real practical payoff: **exiting a scope is $O(1)$** (you just discard/pop one whole hash table), instead of having to scan and delete every entry tagged with that scope from one giant shared table.

2.2 The three-class design

Class	Represents	Key members	Why it exists
<code>SymbolInfo</code>	one symbol	<code>name</code> , <code>type</code> , <code>next</code> (pointer, for chaining)	The atomic unit of information; the <code>next</code> pointer is what turns an array of buckets into a

Class	Represents	Key members	Why it exists
			proper hash table with chaining, rather than an array that breaks on the second collision.
ScopeTable	one scope	array of bucket head-pointers, id (its scope number), parent_scope pointer	Implements the actual hashing (Insert/Lookup/Delete/Print) for exactly one nesting level. The parent_scope pointer is what lets Lookup walk <i>outward</i> toward the global scope when a name isn't found locally.
SymbolTable	the whole program's current nesting state	pointer to current_scope_table	The "stack manager": Enter Scope pushes a new ScopeTable (wiring its parent_scope to the old current one) and Exit Scope pops it. Every operation you expose to the outside world (Insert , Lookup , Remove , Print) is really "do this to whichever ScopeTable is currently on top, walking to parents only for Lookup ."

2.3 The SDBM hash function — mechanics and a fully worked example

You are given **two slightly different-looking implementations** of the exact same underlying idea across your two offline specs. Know both, because a quiz can ask you to spot the difference.

C++ version (from the Symbol Table spec)— reduces modulo the bucket count **on every character**:

```
unsigned int SDBMHash(string str, unsigned int num_buckets) {
    unsigned int hash = 0;
    unsigned int len = str.length();
    for (unsigned int i = 0; i < len; i++)
        hash = ((str[i]) + (hash << 6) + (hash << 16) - hash) % num_buckets;
    return hash;
}
```

C version (from the Lexical Analysis spec, used to match sample output with bucket size 7) — accumulates the **raw** hash across the whole string and does **no modulo inside the loop**; the caller is expected to take `% num_buckets` afterward:

```
unsigned int sdbmHash(const char *p) {
    unsigned int hash = 0;
    auto *str = (unsigned char *) p;
    int c{};
    while ((c = *str++))
        hash = c + (hash << 6) + (hash << 16) - hash;
    return hash;
}
```

What's actually going on mathematically: $(hash \ll 6) + (hash \ll 16) - hash$ is just $hash * 64 + hash * 65536 - hash = hash * 65599$. So both functions are really computing the recurrence

```
hash_i = char_i + 65599 * hash_(i-1)      (Horner's-rule-style polynomial
hash, base 65599)
```

Worked example — hash the 2-character name "ab" into 7 buckets.

'a' = ASCII 97, 'b' = ASCII 98.

Step 1 ($i=0$, char 'a'): $hash = (97 + 0 + 0 - 0) \bmod 7$. Since $97 = 7 \times 13 + 6$, $hash = 6$.

Step 2 ($i=1$, char 'b'): $hash = (98 + (6 \ll 6) + (6 \ll 16) - 6) \bmod 7$. $6 \ll 6 = 384$, $6 \ll 16 = 393216$. Sum = $98 + 384 + 393216 - 6 = 393692$. $393692 = 7 \times 56241 + 5$, so $hash = 5$.

Final bucket index for "ab" with 7 buckets = 5. (If your implementation prints 1-indexed bucket numbers in **P A/P C** output — which the sample outputs like **"at position 2, 1"** suggest — remember to add 1 when displaying, while keeping 0-indexed arrays internally.)

Practical exam advice, stated carefully: because **unsigned int** arithmetic silently wraps at 2^{32} , and the two functions differ in *when* they reduce modulo **num_buckets**, do **not** assume the two are freely interchangeable for arbitrary long strings — always implement the *specific* version given in the spec you're working from, because that's the one the reference/sample output was generated with. If a quiz asks "will these two implementations always produce the same bucket for every string," the safest, defensible answer is: **not guaranteed in general**, because the point at which you reduce modulo a value that doesn't evenly divide 2^{32} can change the result once the accumulator would otherwise have overflowed — so match your implementation to whichever the sample I/O was built from rather than assuming equivalence.

2.4 Collisions and the operation set

Collisions (two different names hashing to the same bucket) are resolved by **separate chaining**: each bucket is the head of a linked list of **SymbolInfo** nodes, and a new entry is appended (or prepended) to that bucket's chain. This is why every insertion/lookup/deletion report in your sample outputs is phrased as **"position B, C"** — **B** is the bucket index and **C** is the entry's position within that bucket's chain (both typically 1-indexed in the printed output).

The seven operations, and what each *actually* has to do underneath:

- **I <name> <type>** — hash **name** to find the bucket in the **current** scope table only; if **name** isn't already present in that bucket's chain, append a new **SymbolInfo** and report success with its **(bucket, chain-position)**; if it's already there, report a collision/duplicate message instead of inserting. Special cases: type **FUNCTION** is followed by its return type and then argument types on the same line; **STRUCT/UNION** types are followed by alternating **(member-type, member-name)** pairs — your **SymbolInfo**'s **type** string effectively has to be able to hold this whole compound description.
- **L <name>** — hash **name** and search the current scope table's bucket chain; if not found, move to **parent_scope** and repeat; report as soon as found, or report "not found in any of the ScopeTables" once you fall off the root.

- **D <name>** — same search as lookup, but **restricted to the current scope only** (you cannot delete a parent's symbol from a child scope) — unless the specific online variant (like **DT**, see §2.5) explicitly asks you to search everywhere.
- **P A** — print every scope table from the current one up through the root, each with only its non-empty buckets.
- **P C** — print only the current scope table.
- **S** — push a brand-new **ScopeTable**, wire its **parent_scope** to the previous current table, make it the new current table, and print **ScopeTable# X created**.
- **E** — pop/destroy the current scope table (print **ScopeTable# X removed**) and make its parent the new current table. **You may never exit the root scope.**
- **Q** — quit, which implicitly performs an **E** for every remaining open scope (root last), so you'll typically see a cascade of "removed" lines right before the program ends.

2.5 Four ways this gets extended — worked, not just listed

Every "online" symbol-table problem you were given is really the same base design above, plus **one new feature bolted on**. Understanding *why* each feature needs the modification it needs is what will actually get you marks, because the quiz-setter's whole trick is to give you a *fifth*, unseen feature built the same way.

(a) Hierarchical scope IDs + a Merge command. Instead of numbering scopes sequentially (1, 2, 3, ...), a scope's ID reflects its position in the *scope tree*: if a parent's ID is **x**, its children are numbered **x.1**, **x.2**, **x.3**, ... in the order they're entered. This just means **ScopeTable** needs to track, in addition to **parent_scope**, **how many children it has created so far**, so the next child knows its own suffix.

The new command **M** ("merge current scope into parent") moves every **SymbolInfo** from the current scope's buckets into the corresponding logic of the parent scope, then deletes the current scope. The one subtlety worth internalizing: **merging can fail**. If a name in the child scope already exists in the parent, that's a genuine conflict — real variables named the same thing in adjoining scopes that are about to become *one* scope. The correct, defensible design (and what the sample output shows) is to print **Merge failed: 'x' already exists in parent ScopeTable and abort the merge without touching either table** — you don't get to silently overwrite the parent's variable or silently drop the child's, because both would lose information the programmer might care about. Marks (10 total) are explicitly weighted toward *doing the move correctly* (3)

and *handling the merge error* (2), which tells you the grader is specifically checking whether you attempted the atomic-or-nothing merge behavior, not just the happy path.

(b) Unused-variable and dead-store detection ("Optimization"). A new command **A** `<var>` assigns a constant to a variable (you don't need to type-check it — assume it exists). Now define two related-but-distinct notions:

- A variable is **unused** if it is *never looked up* (**L**) during its entire lifetime in that scope — checked once, when the scope exits.
- An assignment is a **dead store** if there's no lookup of that variable *between this assignment and the next event that would make the write moot* — either the *next* assignment to the same variable, or the scope closing. In other words: a write is only "live" (not dead) if something reads it before it gets overwritten or thrown away.

These are genuinely different questions ("was it ever read at all?" vs. "was *this particular* write ever read before being clobbered?"), and a variable can trigger *both* reports independently — e.g., an assignment that's immediately followed by another assignment with no intervening read is a dead store *at that point*, and if the variable is *also* never read for the rest of its life, it's separately reported as unused when the scope exits. This is exactly the compiler-optimization concept of **liveness analysis**, just at the granularity of an interpreted command stream instead of a full CFG (control-flow graph) — the marks split (4 for unused-variable reporting, 6 for dead-store reporting) tells you the dead-store logic is the meatier, more error-prone half, so budget your debugging time accordingly.

(c) A toy interpreter built on top of the symbol table ("Mickey Lang"). This variant strips away the "commands are compiler-table operations" framing and instead gives you a tiny integer-only language (**declare**, **set**, **+**, **print**, **{/}** for scoping) — but underneath, it's *the same symbol table* doing the same job: **declare** = **Insert**, **set**/**+**/**print** all need a **Lookup** first, **{/}** are **Enter Scope**/**Exit Scope**. The pedagogical point here is that **a symbol table isn't just a homework data structure — it's the backbone of every interpreter/compiler front-end**, which is exactly why this exercise exists. The one behavioral rule that trips people up: **on any error (redeclaration in the same scope, or use of an undeclared variable), skip only that one line and keep going — never terminate the whole run**. This mirrors the "don't crash on the first syntax/semantic error" philosophy from Part I.

(d) Full-table type deletion (DT) and scope-local symbols (IS). **DT** `<type>` breaks the usual "operations only touch the current scope" rule on purpose: it must walk **every scope table currently on the stack** and delete every symbol of the given type, wherever it lives. This forces you to design your **SymbolTable** class with a way to iterate

over *all* currently-open scopes (e.g., following `parent_scope` pointers from the current scope all the way to the root), not just operate on the top one.

IS `<name> <type>` ("insert scope-local") creates a symbol that's visible **only within the exact scope it was declared in** — not to child scopes (which is normal — child scopes already can't see it via the *parent* pointer chain going the wrong way) and, more unusually, **not to ancestor scopes either**, even though ancestor scopes are exactly what a normal **Lookup** would search when called *from inside* that scope. The trick is that this restriction only matters when **Lookup** is invoked from a **different** scope than the one where the symbol lives — a scope-local symbol should still be perfectly visible to lookups issued from *within its own scope*. Implementation-wise, tag the **SymbolInfo** with an "is scope-local" flag, and when **Lookup** walks up the parent chain (i.e., when it's checking an ancestor on behalf of some descendant scope), it must **skip over** any flagged entries it finds — but not when the scope currently being searched *is* the one that owns the symbol.

2.6 Hygiene rules that show up as short-answer/viva questions

Several specs insist on **no STL** (raw arrays/pointers only) and **compiling with `-fsanitize=address`**. These are not arbitrary — because your symbol-table classes use manual **new/delete** (no STL containers to manage memory for you), it is very easy to leak memory (forget to **delete** a **ScopeTable**'s buckets in its destructor) or to access freed memory (a dangling `parent_scope` pointer after a scope was popped incorrectly). AddressSanitizer instruments the binary at compile time to catch exactly these two classes of bugs — heap-buffer overflows and use-after-free/leaks — at runtime, immediately, instead of them silently corrupting memory and crashing somewhere unrelated later. **If asked "why `-fsanitize=address`?" the answer is: to catch memory leaks and use-after-free/invalid-access bugs that come from manual dynamic memory management, since no STL smart pointers or containers are doing that bookkeeping for you.**

2.7 Symbol-table short-answer bank

Q: What information does a symbol table entry typically hold, and which parts does code generation actually use? A: Typically: name, type, scope/nesting level,

memory location or stack offset, size, and — for arrays — dimensions, or — for functions — return type and parameter list. Code generation specifically needs the **type** (to choose the right instructions/registers — integer ops vs. floating-point ops) and the **memory location/offset** (to emit the correct `MOV [address], ...` or `[EBP-k]` reference).

Q: How are scopes managed in a symbol table? A: Each scope gets its own hash table (a Scope Table); these are organized as a **stack** (or equivalently a linked list via parent pointers). Entering a scope pushes a new Scope Table whose parent pointer is the previous top; exiting pops the current Scope Table. A lookup checks the current Scope Table first, then follows parent pointers outward until the symbol is found or the root is exhausted with no match.

Q: Trace this sequence (hierarchical-ID + merge style) and state the output: S (creates 1.1); I x int; S (creates 1.1.1); I x char; M. A: Entering `1.1.1` and inserting `x` there, then merging `1.1.1` into `1.1` finds that `x` **already exists** in the parent (`1.1`), so the merge **fails**: `Merge failed: 'x' already exists in parent ScopeTable`, and neither table is modified.

Q: What's the difference between a dead store and an unused variable — with an example? A: A dead store is a *write* that gets overwritten (or the scope ends) before ever being read. An unused variable is one that is *never* read during its whole lifetime. Example: `A foo` then immediately `A foo` again with no `L foo` in between — the *first* write is a dead store (because the second write clobbers it before any read). If, additionally, `foo` is never read for the rest of the scope either, it is *also* reported as an unused variable when the scope exits — the two reports are independent and can both fire for the same variable.

PART III — Lexical Analysis with Flex

3.1 What scanning actually is

Lexical analysis converts a flat stream of characters into a stream of **tokens** — pairs of `(token-type, lexeme)` like `<ID, x>` or `<CONST_INT, 5>`. Conceptually, a scanner is a **deterministic finite automaton (DFA)** built by combining the regular expressions for every token type; Flex's job is to generate that DFA (and the C code to drive it) for you from a list of regex-to-action rules, so you never build the automaton by hand. `yyllex()` — the function the generated `lex.yy.c` provides — **is** that

automaton's driver: each call scans forward from where it left off and runs whichever action code is attached to the longest pattern that matched (this is called the **maximal-munch / longest-match rule**, and it is the single most important theoretical idea behind everything Flex does — remember it, because it directly explains several "gotcha" requirements you've been given, see §3.6).

3.2 Anatomy of a `.l` file

A Flex source file has exactly **three sections**, separated by lines containing only `%%`:

1. **Definitions** — `%{ ... %}` blocks are copied verbatim near the top of the generated C file (used for `#includes`, global variable declarations, helper function declarations); plain lines here declare **named sub-patterns** (e.g. `Letter [a-zA-Z]`) that can later be referenced as `{Letter}` inside a rule's regex, and `%option` lines configure the generated scanner (`noyywrap` — always include it and don't worry further; `yylineno` — auto-tracks the current line number, but you must set `yylineno = 1` yourself before the first `yylex()` call; `nodefault` — suppresses Flex's default "echo unmatched input" rule).
2. **Rules** — each line is `pattern { action C++ code }`. **The pattern must start at column 0** — any rule line that starts with leading whitespace is instead treated by Flex as literal code to copy into the generated file, not a pattern, which is a very easy mistake to make. When more than one rule could match at the current position, Flex picks the **longest match**; if two rules match the **same** length, the rule that appears **earlier in the file wins** — so order your specific patterns (like keywords) before your general ones (like identifiers), or a keyword will get swallowed by the identifier rule.
3. **Subroutines** — ordinary C/C++ code copied verbatim, almost always containing `main()`, which points `yyin` at your input file and calls `yylex()`.

`yytext` is the global buffer holding whatever the currently-matched rule actually matched — this is what you inspect/transform inside an action to build the token you print or insert into the symbol table.

3.3 The complete token catalogue

Category	Detail	Token form	Goes into symbol table?
Keywords	if else for while do break int char float double void return switch case default continue goto long short static unsigned	e.g. <IF> (no value needed)	No
Integer literal	one or more digits	<CONST_INT, value>	Yes
Float literal	e.g. 3.14159 , .314159 , 000.0314 , 314159E10 , 3.14159E-10	<CONST_FLOAT, value>	Yes
Char literal	single quoted char, incl. escapes \n \t \\ \' \a \f \r \b \v \0	<CONST_CHAR, ascii-char> (store the <i>actual</i> character/ASCII code, not the escape text)	Yes
Identifier	[a-zA-Z_][a-zA-Z0-9_]* — first char cannot be a digit	<ID, name>	Yes
String literal	double-quoted, can span lines if a line ends in \	<STRING, content> (with escapes resolved)	No
Operators/punctuators	see table below	<TYPE, symbol>	No
Comment (// ... or /* ... */)	logged, but produces no token	—	No
Whitespace	consumed silently	—	—

Operators & punctuators (exact mapping — memorize):

Symbol(s)	Token type
+ −	ADDOP
* / %	MULOP
++ --	INCOP
< <= > >= == !=	RELOP
=	ASSIGNOP
&&	LOGICOP
!	NOT
()	LPAREN RPAREN
{ }	LCURL RCURL — these two additionally trigger Enter Scope / Exit Scope on the symbol table, not just token emission
[]	LTHIRD RTHIRD
,	COMMA
;	SEMICOLON

Worked micro-example. For the source `int x=5;` the scanner should emit, in order: `<INT>` (keyword — no symbol table entry) → `<ID,x>` (identifier — insert `x` into the current scope) → `<ASSIGNOP,=>` → `<CONST_NUM,5>` (integer literal — insert `5` into the symbol table too, per the spec's own worked example) → `<SEMICOLON,;>`.

3.4 Escape sequences and multi-line constructs — the sharp edges

- A char literal like `'\n'` requires you to *read* four raw characters (`'`, `\`, `n`, `'`), but the value you *store* is the actual newline character (ASCII 10), while the *lexeme you print in the log file* should show the literal two-character sequence `\n` (backslash-n), not an actual embedded newline that would break your log's own line structure. The same care applies to `'\\'` — don't let the first backslash be mis-parsed as starting a *different* escape.

- Strings can be **multi-line** if each line except the last ends in a bare `\` immediately before the newline; inside strings, `\n`, `\t`, etc. get converted to their real character values.
- Newlines in the *input file itself* may be `\n` (Unix) or `\r\n` (Windows) depending on where the file came from — handle both, or you'll miscount lines on files created on Windows (relevant since you're developing on an MSI Windows 11 laptop but the assignment likely expects Unix-style line handling on the grading machine).
- A single-line `// comment` that ends its line with `\` **continues onto the next line** as part of the same comment (mirrors the C preprocessor's line-continuation rule).
- An unterminated `/* ... */` comment or an unterminated string, still open when EOF is hit, is a **lexical error** (see §3.5) — this only shows up once your automaton reaches end-of-input still inside the "in a comment"/"in a string" start state, which is exactly why you need start states in the first place (see §3.6).

3.5 The nine lexical error categories

#	Error	Example trigger
1	Too many decimal points	<code>1.2.345.789</code>
2	Ill-formed number	<code>1E10.7</code>
3	Invalid suffix on a number / invalid prefix on an identifier	<code>12abcd</code>
4	Multi-character constant	<code>'abc'</code>
5	Unfinished character	<code>'a</code> , <code>'\n</code> , <code>'\</code> (missing closing quote)
6	Empty character constant	<code>''</code>
7	Unfinished string	closing <code>"</code> never appears
8	Unfinished comment	closing <code>*/</code> never appears before EOF
9	Unrecognized character	any symbol not covered by any rule

Output format is standardized: `Line No. <n>: Token <TOKEN> Lexeme <lexeme> found.` for ordinary tokens/comments, and `Line No. <n>: <error message>.` for errors; the log file ends with the total line count and total error count.

Only **non-empty buckets** are printed whenever the symbol table is dumped, and the reference bucket size used to generate sample output is **7**.

3.6 Why "start states" exist — and the maximal-munch connection

A pure list of independent regex rules can't express **context-dependent** matching — e.g., "the character **]** means one thing normally, but a different thing while I'm still inside a multi-line comment I haven't closed yet." Flex's **start states** solve this by letting you say "this pattern only applies while the scanner is in state X." **%x MYSTATE** declares an **exclusive** state (only patterns explicitly tagged **<MYSTATE>** can match while it's active — everything else is invisible), while **%s MYSTATE** declares an **inclusive** state (ordinary **INITIAL**-level patterns can *still* match too). You switch states from an action using **BEGIN MYSTATE;** and must remember to **BEGIN INITIAL;** once you're done, or the scanner gets permanently stuck treating everything as if it's still inside that construct.

This directly explains the recurring instruction across several of your online problems to **"consume an invalid lexeme as a single unit — never split it into a valid token followed by leftover characters."** This is the maximal-munch principle in its error-handling form: once the scanner has committed to matching, say, a numeric-looking pattern, an invalid suffix on it (**12UU**, **0x2AZ**) must still be swallowed as *one* erroneous lexeme rather than the scanner backing off after a valid prefix and re-scanning the rest as separate (bogus) tokens — because that's what "longest match, one token per match" *means*, applied to the error case.

3.7 Extending the scanner — four case studies

Stack-based structural validation (tag matching). For validating properly nested **<book>**, **<title>**, **<author>** tags: push on every opening tag, and on a closing tag, check it against the **top of the stack** — if it doesn't match, that's a tag mismatch; if the stack is empty when a closing tag arrives, that's an unexpected closing tag; leftover entries on the stack at EOF mean missing closing tags for whatever's still there; an unrecognized tag name is reported as unsupported. **This is precisely the "balanced parentheses" problem from formal language theory**, which not coincidentally is also

the canonical example of something a *finite automaton (regular language)* cannot recognize in general but a *stack-based automaton (context-free language)* can — which is exactly why the assignment tells you to add a stack on top of Flex rather than trying to solve it with regex alone, and it's a nice bridge into why Part IV needs a proper grammar-based parser instead of "just Flex" for anything with real nesting.

Stateful key/value grammar validation (a simplified Python dictionary). You track, via start states, whether the scanner currently expects a *key*, a *value*, or is *inside a quoted string*, with a counter (or your existing stack machinery) tracking how deeply nested the `{ }` are. The required behavior — **report only the first error found** — differs from the tag-matching problem's similar "stop at first error" but matters because *which* error message applies depends on exactly which state you were in when things went wrong (invalid key vs. missing separator vs. unmatched brace).

Extending the numeric-literal grammar (octal/hex/suffixes). Octal constants start with `0` plus at least one more digit, restricted to `0–7`; hexadecimal constants start with `0x/0X` plus at least one hex digit. Numeric suffixes (`U`, `L`, combined `UL/LU` in any case, `F` for float) attach additional token types (`CONST_UINT`, `CONST_LONG_INT/CONST_LONG_FLOAT`, `CONST_ULONG`, `CONST_FLOAT`). Every one of these problems repeats the same instruction from §3.6: **an invalid or repeated suffix must be consumed as one lexeme and reported as one error — never split.**

Off-side-rule scanning (Python-style indentation). Tracking `INDENT/DEDENT` tokens from leading whitespace is a textbook example of what's sometimes called the "**off-side rule**" (whitespace-sensitive syntax, as in Python or Haskell) — and it's fundamentally different from every other lexical feature you've built, because it requires **remembering state across entire lines**, not just within a single token. You maintain a **stack of indentation widths**, starting with `[0]`. A new line's leading-space count that's *greater* than the stack's top pushes the new width and emits one `INDENT`; a count that's *smaller* pops levels one at a time, emitting one `DEDENT` per pop, until the top matches (if it never exactly matches an existing stack level, that's an **inconsistent dedentation** error, and — per this problem's stated rule — scanning stops entirely at that point, no further tokens or dedents are emitted for that line); a count *equal* to the top emits nothing. At end-of-file, emit enough `DEDENT`s to unwind the stack back to `0`. Blank lines and comment-only lines are simply skipped for indentation-tracking purposes (they don't reset or change anything).

PART IV — Syntax and Semantic Analysis with ANTLR4

4.1 Why lexical analysis alone isn't enough

Regular expressions (and therefore Flex/DFAs) can recognize *flat* patterns, but they fundamentally **cannot count arbitrarily deep nesting** — a regex can't verify that every `{` in a whole C program has a matching `}` in the right place, because that requires unbounded memory of "how many are currently open," which a DFA's fixed number of states cannot provide (this is the same idea that came up with the tag-matching lexical exercise, just at the scale of a whole program instead of a few tag names). **Context-free grammars**, parsed with a stack-based algorithm, can. That's the conceptual reason your pipeline moves from Flex (regular) to ANTLR4 (context-free) for anything involving real program structure — statements, expressions, function bodies, nested blocks.

4.2 ANTLR4 grammar anatomy

- A grammar file starts with `grammar Name;` (or `lexer grammar XLexer; / parser grammar XParser;` if split into separate lexer and parser grammars, the latter importing the former's tokens via `options { tokenVocab = XLexer; }`).
- **Rule naming convention is meaningful, not cosmetic:** a rule starting with a **lowercase** letter is a **parser rule** (defines syntax — how tokens combine); a rule starting with an **uppercase** letter is a **lexer rule** (defines a token). ANTLR generates the lexer directly from your uppercase rules — you don't hand-write a separate scanner the way you did with Flex.
- Running the `antlr4` tool on your `.g4` file(s) generates, among others: `<Name>Lexer`, `<Name>Parser`, `<Name>BaseVisitor` / `<Name>Visitor`, `<Name>BaseListener` / `<Name>Listener`, and a `<Name>.tokens` file (which is the ANTLR analogue of Bison's `y.tab.h` — it's where you'd look to see the actual integer values assigned to each token type).
- Grammar rule alternatives are separated by `|`. You can **label an alternative** with `#Name` so that, when generating a Visitor, ANTLR produces one `visitName(...)`

method per labeled alternative instead of one big method for the whole rule with a manual `if` chain inside it — this is very useful for expression grammars where each operator gets its own alternative.

4.3 Ambiguity, precedence, and ANTLR4's headline feature: direct left recursion

Classical top-down (LL) parsers **cannot handle left-recursive rules directly** — a rule like `expr : expr '+' expr | INT ;` would send a naive recursive-descent parser into infinite left-recursion before it ever consumes a token, because it tries to expand `expr` by first trying to match `expr` again with nothing consumed yet. Traditional LL grammars had to be manually rewritten (left-factored, precedence climbing via a tower of `expr`, `term`, `factor` rules) to avoid this.

ANTLR4's actual, book-headline feature is that it supports *direct* left recursion natively — you're allowed to write the natural, ambiguous-looking grammar:

```
expr : expr '*' expr
      | expr '+' expr
      | INT
      ;
```

and ANTLR automatically rewrites it internally into the equivalent unambiguous precedence-climbing form. **Precedence is expressed purely by the order of the alternatives** — the alternative written *first* binds tighter (higher precedence). Associativity defaults to *left*, but you can force **right-associativity** with an option on that alternative: `expr : <assoc=right> expr '=' expr | ... ;`.

This maps directly onto your ICG "chain of assignments" problem. The requirement that `a = b = c = expr` be **right-associative** is *exactly* the `<assoc=right>` case for the assignment alternative in a left-recursive `expr` rule — if you're asked a conceptual question about why chain-assignment is right-associative or how a grammar expresses that, this is the mechanism.

The classic **dangling-else ambiguity** (`if (c1) if (c2) s1; else s2;` — which `if` does the `else` belong to?) is resolved the same "alternatives resolved by order/greediness" way: ANTLR's adaptive prediction, when both interpretations are

possible, prefers the alternative that lets it consume more input consistently with the standard C rule (else binds to the nearest unmatched if) — practically, you write the `if...else` alternative in a way that ANTLR's default greedy resolution naturally attaches the `else` to the innermost `if`, matching the spec's own instruction to "resolve if-else conflicts using ANTLR's precedence rules."

4.4 Listener vs. Visitor — and why your assignment mandates Visitor

ANTLR gives you two different ways to walk the parse tree it built:

- **Listener** — you implement `enterX()/exitX()` callback methods for each rule, and a built-in `ParseTreeWalker` automatically calls them as it walks the *entire* tree for you, top to bottom. You never call anything yourself; the walker drives everything. The catch: enter/exit methods **return nothing** — you communicate results only via side effects (fields, external maps), not return values.
- **Visitor** — you implement `visitX()` methods yourself, and *you* are responsible for calling `visit(child)` (or `visitChildren(ctx)`) inside each method to recurse manually. Because you're calling `visit()` yourself, `visitX()` **can return a value**, which flows back up to whichever method called it.

Why your grammar assignment specifically requires the Visitor pattern and disallows actions embedded in the grammar file itself: code generation is inherently a **bottom-up, value-returning** process — to generate code for `a + b`, you need the generated code *and address/register* for `a`, and the generated code *and address* for `b`, *before* you can combine them into the code for the whole expression. The Visitor pattern's return values give you this naturally (each `visitExpr()` call returns something like a `{code, addr}` pair that the caller combines); the Listener pattern would force you to stash intermediate results into external maps keyed by context node, which is exactly the kind of bookkeeping the Visitor pattern's return values eliminate. Banning embedded grammar actions on top of that keeps the grammar file **purely about syntax**, with all semantic/code-generation logic cleanly separated into C++ visitor methods — which also means the same parse tree could, in principle, be walked by a *different* visitor for a *different* purpose (e.g., one visitor pass to populate the symbol table, a second visitor pass to check semantics, a third to generate code) without ever touching the grammar file. This separation-of-concerns argument is exactly the kind of "why" a conceptual quiz question is fishing for.

4.5 Semantic predicates (brief, but a real ANTLR4-book topic)

A **semantic predicate** is a boolean C++/Java expression written as `{condition}?` embedded in a rule, used either to **validate** ("this alternative is syntactically fine, but only accept it if `condition` holds, else raise an error") or to **disambiguate** ("of several alternatives that could all syntactically match here, only *try* this one if `condition` holds" — evaluated during ANTLR's prediction phase, before committing to a parse). This is distinct from a semantic **action** (`{...}` with no `?`) which just runs code and doesn't affect parsing decisions.

4.6 Error handling strategy

ANTLR4's default (`DefaultErrorStrategy`) tries to **recover and keep parsing** after a syntax error, using single-token insertion or deletion heuristics to resynchronize with the grammar — this is philosophically the same "don't just crash, keep going and report more problems" idea from Part I. If you want a parser that gives up immediately on the first error instead (useful when ANTLR is embedded as a small step inside a larger tool and you don't want it burning time on hopeless input), you can swap in `BailErrorStrategy`. Practically, in your assignment you report syntax errors yourself via well-formed messages using `ctx->getStart()->getLine()` to get the offending line number — the ANTLR-generated recovery machinery keeps the parser alive long enough for you to keep finding *more* errors in the same run, rather than stopping at the first one.

4A. Building a Symbol Table "the ANTLR Way" — straight from the reference book

The Definitive ANTLR 4 Reference dedicates a full chapter specifically to this, because it's such a common real task, and it maps almost exactly onto what your Assignment 3 asks you to do. The book's pattern is worth knowing on its own terms:

- Define a small class hierarchy: a **Symbol** (name + type, exactly like your `SymbolInfo`), and a **Scope** interface with `define(symbol)`, `resolve(name)`,

and `getEnclosingScope()`. Concrete scope kinds — `GlobalScope`, `LocalScope` — extend a common `BaseScope` implementation, mirroring your `ScopeTable/SymbolTable` split.

- **The key idiom the book teaches, and the one most worth remembering for a conceptual question:** because ANTLR's parse-tree context objects are meant to represent *syntax*, not accumulate arbitrary extra bookkeeping fields, the book attaches "which scope is active at this tree node" information using a **side map** (`ParseTreeProperty<Scope>`) from context node → `Scope`, rather than storing a `Scope` field directly inside every generated context class. If asked *why* you'd do this instead of just adding a scope pointer field to your visitor's context objects: it keeps the auto-generated parse-tree classes untouched and reusable, and lets *different* visitor passes attach *different* kinds of metadata to the same tree without interfering with each other.
- **The two-pass pattern:** rather than trying to define *and* check symbols in one single walk, the book's worked example uses **two separate visitor/listener passes** over the *same* tree: a first "definition" pass that walks the tree purely to create scopes and register every declared symbol into the right scope; then a second "reference/resolution" pass that walks the tree *again*, this time looking up every identifier *use* against the scopes built in pass one, reporting undeclared-variable and similar errors. **Why two passes instead of one?** Because a language may allow a name to be *used* before its point of declaration is textually reached in a naive single left-to-right pass (forward references — think of calling a function that's declared later in the file, which your own Assignment 3 spec explicitly allows: "a function needs to be defined or declared before it is called" is actually a *constraint* you must check, which itself implies you need to know about *all* declarations, not just ones seen so far, in some cases) — building the *complete* symbol picture first, then checking references against the complete picture, avoids spurious "undeclared" errors for perfectly legal forward-referencing code, and is a strictly more robust design than trying to catch everything in one pass. This two-pass idea is precisely why your own assignment inserting symbols "in the parser's variable declaration rule" and then doing uniqueness/undeclared-variable checks are conceptually **separate concerns**, even if your implementation interleaves them within one visitor for simplicity.
- **This is also the conceptual justification for why symbol-table insertion moved from the *lexer* (Assignment 2) to the *parser* (Assignment 3):** a lexer sees identifiers as isolated tokens with no idea whether that occurrence is a *declaration* or a *use* — `int x;` and `x = 5;` both contain the token `<ID,x>`, and only the *parser*, which understands the surrounding grammatical context (a

declaration rule vs. an expression rule), can tell which one is happening. Symbol table population is therefore fundamentally a **syntax-aware** operation, which is exactly why it belongs in the parser (or in a listener/visitor over the parser's output), not the lexer.

4B. Building Interpreters and "Bytecode Compilers" — the book's other big relevant chapters

The ANTLR4 book's later chapters walk through building (1) a direct tree-walking **interpreter** (a visitor that *evaluates* expressions immediately as it visits them — no code is emitted, the answer is just computed on the spot) and then (2) a proper **bytecode compiler**, which instead emits instructions for a small **stack-based virtual machine** (opcodes like "push a constant," "load a variable," "add the top two stack values," "print the top of stack," etc.) rather than executing anything immediately.

This is conceptually the *exact same task* as your Assignment 4 (ICG), just targeting a different kind of machine — and it sets up a genuinely useful **conceptual compare-and-contrast** that a quiz might reasonably ask about, since it's straight out of the assigned reference text:

	Stack-based (0-address) code — book's bytecode VM	Register-based (your x86/FASM target)
Operands	Implicit — always "the top of the stack"	Explicit — named registers (EAX , ECX , EDX , ...)
Instruction shape	PUSH , POP , ADD (pops 2, pushes 1 result)	MOV , ADD reg, reg , needs explicit source/destination
Code size	Usually more compact, very simple to generate — every sub-expression just pushes its result	Usually fewer instructions executed, but code generation must actively manage <i>which register holds what</i>
Register allocation problem	Doesn't exist — there's no finite register file to run out	A real, hard problem — exactly why the "minimum number of temporaries"

	Stack-based (0-address) code — book's bytecode VM	Register-based (your x86/FASM target)
	of, the stack grows as needed	optimization question (§5.5 / old Quiz Q18) exists at all
Typical use	Easy first target for a compiler course / real VMs like the JVM	What a real native compiler must eventually produce

If you're asked "why does the book target a stack machine while your course targets x86" — the honest, good answer is: a stack machine is *pedagogically* simpler because code generation for it requires no register-allocation decisions at all (every value just goes on/comes off the stack), which is why it's the natural *first* code-generation target to teach the *concept* of intermediate code generation with. Targeting real x86 registers, as your course does, adds the *additional, genuinely hard* problem of deciding which value lives in which register and for how long — which is precisely the material in §5.5 below (dead code / minimum temporaries), and is a big part of why real compilers spend so much effort on register allocation as its own separate compiler phase.

4.7 The complete semantic-check catalogue for your CSubset language

Each bullet below is a real rule from your Assignment 3 spec, with the *reasoning* spelled out, since "explain why this is a semantic error, not a syntax error" is a natural quiz framing (the distinction being: syntax errors are about **shape** — is this even a legal-looking sentence in the grammar; semantic errors are about **meaning** — is this grammatically-legal sentence *actually consistent*, which requires knowing about types and declarations that the grammar alone can't express).

- **Assignment type consistency** — the two sides of `=` must be type-compatible; this can't be a syntax rule because *any* expression is syntactically legal on the right of `=` — only by knowing the *type* of that expression (which requires symbol-table lookups and recursive type inference over the expression tree) can you tell if it's wrong.
- **Array index must be an integer** — `a[x]` is syntactically fine whatever `x` is; only semantic analysis, consulting `x`'s declared type, can catch `a[y]` where `y` is a `float`.

- **Both operands of % must be integers** — same reasoning; modulus is undefined for floats in this language.
- **Function call argument consistency** — the number *and* type of arguments actually passed must match the function's declared parameter list — this needs the function's **SymbolInfo** to carry its full parameter-type list, which is why the spec tells you to extend **SymbolInfo** with function-specific fields (§4.9).
- **A void function cannot be used inside an expression** — `x = foo();` is only legal if `foo` returns a value; the grammar can't distinguish "used as a statement" from "used as a sub-expression" contexts without help, so this is checked in the visitor by knowing the calling context.
- **Type conversion** — assigning a float value to an int variable should produce an error/warning (implicit narrowing); the results of relational (**RELOP**) and logical (**LOGICOP**) operators are always typed as `int` (this is the classic "booleans are just integers" C convention).
- **Uniqueness checking** — (a) every identifier *used* in an expression must have been declared somewhere visible (an undeclared-variable check, exactly the "reference pass" from §4A), and (b) no two declarations may reuse the same name **in the same scope** (a redeclaration check, exactly the "definition pass" from §4A catching a duplicate `define()` call).
- **Array-vs-scalar consistency** — an identifier declared as an array must always be indexed when used, and a non-array identifier must never be indexed — checking both directions.
- **Function declaration/definition matching** — if a function is both declared (prototype) and later defined (with a body), the two must agree on return type, parameter count, parameter order, and parameter types; a function call cannot target a non-function identifier at all.

4.9 Extending **SymbolInfo** for functions

Because a plain `(name, type)` pair can't describe "the function `foo` returns `int` and takes `(int, float)`," the spec asks you to add fields — return type, parameter list/types, parameter count — either directly on **SymbolInfo** or on a small helper class referenced from it (the spec's own suggestion, to avoid bloating every non-function symbol with unused function-only fields). This is a direct, practical instance of the general OOP principle "don't force every object in a hierarchy to carry fields only some subset of them need" — a clean design uses composition (a pointer to an optional **FunctionInfo** object) rather than putting every possible field on the base class.

4.10 Syntax/Semantic short-answer bank

Q: Why can ANTLR4 handle a rule like `expr : expr '+' expr | INT ;` directly, when a classic recursive-descent (LL) parser cannot? A: A naive top-down parser trying to expand `expr` would immediately try to match `expr` again before consuming any input token, recursing forever without making progress. ANTLR4 special-cases *direct* left recursion at grammar-generation time, automatically rewriting such rules into an equivalent, non-left-recursive precedence hierarchy internally, so you get to *write* the natural, precedence-order-encodes-priority grammar while ANTLR handles the actual non-left-recursive parsing underneath.

Q: Why does symbol table insertion happen in the parser (Assignment 3) rather than the lexer (Assignment 2)? A: The lexer only sees isolated tokens and cannot tell whether an identifier occurrence is a *declaration* or a *use* — that distinction depends on surrounding grammatical context (e.g., appearing right after a type keyword in a declaration rule vs. appearing inside an expression), which only the parser understands.

Q: Why must the grammar avoid embedded actions, using the Visitor pattern instead? A: It keeps the grammar file purely syntactic and separates semantic logic (type-checking, code generation) into visitor methods, so the same parse tree can be reused by multiple independent visitor passes (e.g., symbol-table population, semantic checking, code generation) without touching the grammar, and because code generation needs *return values* bubbling up from child nodes to parents, which the value-returning Visitor pattern supports directly and the side-effect-only Listener pattern does not.

PART V — Intermediate Code Generation: From Parse Tree to x86 Assembly

5.1 What "intermediate" buys you, and the two models

Generating an intermediate representation instead of jumping straight from a parse tree to final machine code buys you: (a) a simpler, more uniform target for the code-generating visitor to emit (arithmetic expressions, one operator at a time, rather than trying to reason about an entire complex statement in one shot), and (b) a natural point to run **machine-independent optimizations** before worrying about a specific CPU's instruction set. See §4B above for the stack-machine vs. register-machine comparison — your assignment targets the *register* model (real x86), which is the harder, more realistic one.

5.2 The x86/FASM conventions you must know cold

Program skeleton:

```
format ELF executable 3
entry main
segment readable writeable
    a dd 1 DUP (0)           ; global, no initializer -> defaults to 0
    b dd 1 DUP (10)          ; global, initialized to 10 at declaration
segment readable executable
main:
    PUSH EBP
    MOV EBP, ESP
    SUB ESP, 4                ; one SUB ESP,4 per local variable, in
declaration order
    ...
    ADD ESP, <total-locals-bytes>
    POP EBP
    MOV EAX,1
    XOR EBX, EBX
    INT 0x80
    POP EBP
    RET
```

- **Globals** live in the writeable data segment, declared `name dd 1 DUP(initial-value)`, and are referenced everywhere else simply as `[name]`.
- **Locals** live on the stack frame, referenced as `[EBP-4]`, `[EBP-8]`, ... assigned in **declaration order** — each new local variable claims the next multiple of 4 bytes below `EBP` and gets its own `SUB ESP,4` when it comes into scope.
- Numbered `; Line N` comments and sequentially-incrementing `.Lk:` labels accompany essentially every statement — the labels aren't only for branch targets, they're emitted structurally throughout, which is also what makes `goto`/label support possible (see §5.4).

Expression evaluation convention — and why each rule exists:

- Load a value into **EAX** as the "current working register." To combine two sub-expression results, evaluate the first into **EAX**, then **PUSH EAX** before evaluating the second — *because* evaluating the second sub-expression will itself need to use **EAX** (and possibly **ECX/EDX**) as scratch space, which would clobber the first result if you didn't save it first. After the second operand lands in **EAX**, **MOV EDX/ECX, EAX** to get it out of the way, then **POP EAX** to bring the first operand back, and combine the two registers with the actual operator instruction.
- **Multiplication:** **MOV EAX, op1 ; CWD ; IMUL ECX** (or **MUL ECX** for the unsigned case — some sample outputs use one, some the other; always match the specific reference output you're given, and default to the signed **IMUL** since the language's ints are signed). **Why CWD first?** **CWD** sign-extends whatever's in **AX/EAX** into **DX:AX/EDX:EAX**, which the multiply/divide instructions require as their full-width input — skipping it leaves **EDX** holding garbage from a previous computation, corrupting the high half of the result (and catastrophically corrupting the *quotient/remainder* for division, see next point).
- **Division/modulus:** **MOV EAX, dividend ; CWD ; IDIV ECX → quotient ends up in EAX, remainder in EDX**. This is a hardware convention of the **IDIV** instruction itself (it always divides the 64-bit value in **EDX:EAX** by the operand and splits the result this way) — you don't get to choose it, which is exactly why **%** (modulus) is implemented as "do a division and just keep **EDX** instead of **EAX**."
- **Unary minus / bitwise NOT:** **NEG EAX / NOT EAX** respectively — note these are genuinely different operations (**NEG** computes two's-complement negation, **NOT** flips every bit — **NOT** of a value is *not* the same as **NEG** of it, a classic conceptual trap: **NOT x = -x - 1**).
- **Bitwise AND/OR:** **AND EAX,EDX / OR EAX,EDX** once both operands are in registers, following the same push/pop-to-preserve-the-first-operand pattern as arithmetic.
- **Post-increment (x++):** the *old* value must be the one that participates in any surrounding expression, so the generated pattern loads the old value, **PUSHes** it, then increments and stores the new value back to **x**'s memory location, and finally **POP**s the old value back into **EAX** for the surrounding expression to use — this ordering is exactly what makes post-increment *post*.
- **Relational operators (==, <, etc.):** **CMP EAX,EDX** followed by the matching conditional jump (**JE, JNE, JL, ...**) to a "set result to 1" label, with an unconditional fallthrough/**JMP** to a "set result to 0" label, both converging on a shared continuation label — because x86 relational instructions produce **flags**, not a 0/1 integer value

directly, you need this jump-based dance to materialize an actual `int` result usable elsewhere in the expression.

- **Logical `&&/||`**: implemented as a *chain* of `CMP ...,0` + conditional-jump tests, one per operand, rather than a single instruction — this is the assembly-level shape of **short-circuit evaluation** (though note: whether the generated code actually *skips* evaluating the second operand when the first already determines the answer, versus always evaluating both and then combining, is worth checking against your specific sample output before assuming true short-circuiting).
- **`println(expr)`**: evaluate `expr` into `EAX`, then `CALL print_number`.

5.3 A fully worked trace, start to finish

Take this tiny program:

```
int a, b;
int main(){
    int x;
    a = 10;
    b = a + 5;
    x = a == b;
    println(x);
    return 0;
}
```

Symbol table state after declarations: global scope holds `a` and `b` (both `INT`, addressed as `[a]/[b]` in the data segment); `main`'s local scope holds `x` (addressed as `[EBP-4]`).

Generated skeleton pieces, in order:

```
segment readable writeable
    a dd 1 DUP (0)
    b dd 1 DUP (0)
segment readable executable
main:
    PUSH EBP
    MOV EBP, ESP
    SUB ESP, 4                ; local x

.L1:
    MOV EAX, 10                ; Line: a = 10;
    MOV [a], EAX

.L2:
    MOV EAX, 5                  ; Line: b = a + 5; (evaluate "5" first or "a"
first is a design choice – evaluate left-to-right: a, then 5)
```

```

; (following the push/pop convention from 5.2:)
MOV EAX, [a]
MOV EDX, EAX
MOV EAX, 5
ADD EAX, EDX
MOV [b], EAX

.L3:
MOV EAX, [a]           ; Line: x = a == b;
MOV EDX, EAX
MOV EAX, [b]
CMP EAX, EDX
JE .LtrueX
JMP .LfalseX

.LtrueX:
MOV EAX, 1
JMP .LdoneX

.LfalseX:
MOV EAX, 0

.LdoneX:
MOV [EBP-4], EAX

.L4:
MOV EAX, [EBP-4]      ; Line: println(x);
CALL print_number

.L5:
MOV EAX, 0             ; Line: return 0;
JMP .Lend

.Lend:
ADD ESP, 4
POP EBP
MOV EAX, 1
XOR EBX, EBX
INT 0x80
POP EBP
RET

```

(Label names/exact numbering are illustrative — match whatever your own visitor's label counter produces — but the **shape and ordering** above is exactly the convention seen across all six of your ICG sample outputs, and being able to reproduce this kind of trace by hand from a short snippet is a completely realistic long-answer quiz question.)

5.4 Extending code generation — four case studies

Initialization during declaration. For a global (`int a=10;`), the initial value is baked directly into the data-segment line: `a dd 1 DUP (10)`. For a local (`int a=10;` inside a function), there's no equivalent "initialized stack slot" concept in x86 — the stack frame is just raw memory — so instead you emit an ordinary assignment (`MOV EAX, 10 / MOV`

[EBP-k], EAX) immediately after that local's SUB ESP, 4, at the exact point the declaration occurs.

Right-associative chain assignment (a = b = c = expr). Because the language is right-associative here, the *rightmost* expression is the "real" computation — evaluate it **exactly once**, then MOV that single resulting value into a, b, and c in turn. The key insight (and a good conceptual-question target): naively translating this as "assign c=expr, then b=c, then a=b" would still be *correct* semantically for this all-integer-copy case, but it's wasteful (extra loads) and doesn't generalize once expressions can have side effects (e.g., if expr contained something like d++) — evaluating once and fanning the single result out to every target is both more efficient and semantically the only fully correct approach in general.

goto/labels. A label declaration (L1:) becomes an assembly label at that exact program point, and goto L1; compiles to JMP L1. The genuinely interesting wrinkle: the destination can appear **either before or after** the goto in the source. A simple single left-to-right pass can't emit a JMP to a label it hasn't seen yet without knowing what to call it — this is the classic **forward-reference problem** that real assemblers and linkers also have to solve (it's exactly why assemblers traditionally do **two passes**: pass one just records where every label will end up, pass two emits the actual jump targets now that they're all known). In your ANTLR-based setup, you get this almost for free: because you already have the *entire* parse tree in memory before you start generating any code, you can pre-scan it once for label declarations (building a name→internal-label mapping) before your main code-generation visitor pass runs — so "two passes over the tree" (or one pre-pass plus the main pass) is the natural solution, directly analogous to the two-pass symbol-table pattern from §4A.

Compound assignment (+= -= *= /= %=) and bitwise operators (& | ~).

Compound assignment is defined by simple rewriting: x op= expr behaves as x = x <op> (expr) — read x's current value, evaluate expr, apply the same code-generation pattern as the corresponding binary operator, store back to x; it sits at the same precedence level as plain =. The bitwise operators have their own precedence tier, described exactly as: ~ (NOT, right-associative, unary, so it binds tightest) > & (AND, left-associative) > | (OR, left-associative, binds loosest among these three) — and the whole tier sits **above** assignment operators but **below** logical &&/|| in overall precedence. This ordering isn't arbitrary; it mirrors real C's own precedence table, so if you ever forget the exact numbers, recalling "this behaves just like real C's bitwise-vs-logical-vs-assignment ordering" will get you there.

5.5 Optimization: dead code and minimizing temporaries

At the assembly level, the *same* dead-store idea from Part II reappears in a stricter form: a store to a temporary is dead if it's provably never read before being overwritten or the block ends, **given that every non-temporary variable is assumed live at the end of the block** (the problem's own stated assumption — you don't get to assume a real variable is dead just because this particular block doesn't read it again, since some later block might). The general method for these "strike out dead lines" / "minimum number of temporaries" style questions:

1. List the block's instructions in order; for each temporary, find where it's **defined** (written) and its **last use** (read) before either being overwritten or the block ending.
2. If a temporary is redefined before it's ever read even once, its *original* definition is provably dead — strike it.
3. A temporary's **live range** spans from its definition to its last use. Two temporaries whose live ranges **don't overlap** can safely share the same physical storage (a register or a stack slot) — this is the seed idea of real **register allocation**.
4. The **minimum number of temporaries** needed for the whole block equals the **maximum number of live ranges that are simultaneously alive at any single point** in the instruction sequence — conceptually the same problem as interval-graph coloring (find the maximum "overlap depth" across all the intervals).
5. Also look for pure **peephole** redundancy — e.g., a **PUSH EAX** immediately followed by a **POP EAX** with nothing in between that reads or changes **EAX** is a complete no-op pair that can simply be deleted; you'll notice patterns exactly like this in the raw sample outputs shown in §5.3, which is a strong hint that this kind of peephole cleanup is a realistic exam ask.

PART VI — General Compiler Theory Quick-Reference (pure MCQ ammo)

- **Phase order:** Lexical → Syntax → Semantic → Intermediate Code Generation → (machine-independent) Optimization → Code Generation (machine-dependent).
- **Machine-independent:** Lexical, Syntax, Semantic, ICG, high-level Optimization, Symbol Table management.

- **Machine-dependent:** target-specific Optimization, final Code Generation.
- **yyllex():** performs scanning; called repeatedly by the parser to fetch the next token.
- **Tokens never inserted into the symbol table:** keywords, operators/punctuators, strings, comments. **Tokens that are inserted:** identifiers, integer/float/char constants.
- **What a symbol table stores / what code-gen uses from it:** name, type, scope, memory offset, size, (functions) parameter info — code-gen specifically needs type (instruction/register selection) and memory location (addressing).
- **{ / }** in lexical analysis: trigger Enter Scope / Exit Scope on the symbol table, in addition to being tokenized as **LCURL/RCURL**.
- **Bison legacy trivia** (still fair game as general parser-construction knowledge even though you use ANTLR): **%token** declares a terminal; **%type** declares a **nonterminal's** value type (a common trap — a symbol declared via **%type** is *not* a terminal); **-d** generates **y.tab.h**; **YYSTYPE** is the union type of all possible token/nonterminal semantic values; **\$\$,\$1,\$2** reference the rule's own value and its right-hand-side symbols' values positionally.
- **ANTLR generated artifacts (the modern equivalents):** **<Name>Lexer**, **<Name>Parser**, **<Name>BaseVisitor/<Name>Visitor**, **<Name>BaseListener/<Name>Listener**, **<Name>.tokens**.
- **-fsanitize=address:** catches memory leaks and use-after-free/invalid-access errors — relevant because these assignments use manual **new/delete**, not STL containers, to manage scope tables.

PART VII — The 2019 Previous Quiz, Fully Solved and Explained

(Original format: Full Marks 50, Time 50 minutes, 10 MCQ @ 1 mark + 8 short questions @ 5 marks. Note: this quiz predates your course's switch to ANTLR4, so a few questions are phrased in Bison/YACC terms — see Part IV's translation notes for the modern equivalents. The original PDF's answer key was embedded as circled/highlighted marks that don't extract as plain text, so the answers below are independently derived from compiler theory — cross-check with your own course materials/instructor if any differ.)

1. Which are machine-independent phase(s)? i. Symbol table creation, ii. Syntax analysis, iii. ICG, iv. Lexical analysis. Answer: (a) All of the mentioned. All four are

front-end/IR-level activities that don't depend on the target CPU — see Part VI.

2. Which phase(s) are more likely to write to a symbol table? i. Code optimization, ii. Syntax analysis, iii. Semantic analysis, iv. Lexical analysis.

Answer: (c) ii., iii., and iv. Lexical analysis records identifiers as they're first seen; syntax analysis recognizes declarations; semantic analysis fills in/validates type and attribute information. Code optimization typically only *reads* the table (to know a variable's type/size for optimization decisions), it doesn't create new symbol entries.

3. What is the function of `yyllex()`? Answer: (a) scanning.

4. During lexical analysis, which was NOT inserted into the symbol table? (a) integer literals (b) keywords (c) character literals (d) operators. Answer: (b)

keywords is the classic textbook answer (a keyword can never be mistaken for a user symbol name, so it's never stored) — though note that, per your own course's specific token table, operators are *also* excluded; if this exact question appears again, "keywords" is the safest single choice, but be ready to explain that operators are excluded too.

5. In Bison files, `YYSTYPE` indicates: Answer: (b) Data type of token (more precisely, the union type of possible semantic values attached to tokens/nonterminals).

6. `c : a '+' b { $$ = $1 + $2; }` — what happens when matched?

Answer: (a) a and b are added and saved in c. ($\$1 \rightarrow a$, $\$2 \rightarrow b$, $$$ \rightarrow c$, positionally, by rule-symbol order.)

7. `bison -d -y -v your_rol1.y` — which flag generates `y.tab.h`? Answer: (a) `-d`.

8. Where do you see the actual numeric values assigned to your declared tokens? Answer: (a) `y.tab.h`. (ANTLR4 equivalent: the generated `.tokens` file.)

9. Correct translation of `a[1]=10` (2-byte ints, byte-addressable machine):

Answer: (a) — `MOV t2,10 / MOV t1,1 / MOV BX,t1 / ADD BX,BX / MOV AX,t2 / MOV a[BX],AX`. The `ADD BX,BX` step is essential — it scales the index by the element size (2 bytes); options that omit this scaling, or that read instead of write at the end, are wrong.

10. Given `%token T_A; %type T_B; %left OP_A; %right OP_B;` — which statement is FALSE? Answer: (c) "`T_B` is a terminal symbol" is FALSE. `%type`

declares the semantic-value type of a **nonterminal**, not a token — `T_B` is a nonterminal, not a terminal, making (c) the false statement. (Note: (b) "`OP_A` has lower precedence

than `OP_B`" is actually *true*, since precedence increases with each successive `%left/%right` declaration line, and `OP_B` is declared after `OP_A`.)

11. What is stored in a symbol table, and what does code generation use from it?

(5 marks) Model answer: Stored: symbol name, type, scope/nesting level, memory location/offset, size, and (for arrays) dimension info or (for functions) return type and parameter list. Code generation specifically consults **type** — to select the correct instruction/register class (integer vs. float operations) — and **memory location/offset** — to emit the correct addressing (`[name]` for globals, `[EBP-k]` for locals).

12. Did you manage scopes in your symbol table lab? How are scopes generally managed?

(5 marks) Model answer: Yes — one hash table (Scope Table) per scope, organized as a stack (or a parent-pointer list). Entering a scope pushes a new Scope Table whose parent pointer references the previous current table; exiting pops it. Lookup checks the current table, then walks up parent pointers toward the root until the symbol is found or the chain is exhausted.

13. Write a Flex program matching a password: alphanumeric+symbols, ≥1 capital letter, ≥1 digit, ≥1 of `~!@#$%^`.

(5 marks) Model answer:

```
%%
[A-Za-z0-9~!@#$%^]+ {
    int hasUpper = 0, hasDigit = 0, hasSymbol = 0;
    for (char *p = yytext; *p; p++) {
        if (isupper((unsigned char)*p)) hasUpper = 1;
        else if (isdigit((unsigned char)*p)) hasDigit = 1;
        else if (strchr("~!@#$%^", *p)) hasSymbol = 1;
    }
    if (hasUpper && hasDigit && hasSymbol)
        printf("Valid password: %s\n", yytext);
    else
        printf("Invalid password: %s\n", yytext);
}
.|\\n ; /* ignore anything else */
%%
```

Regex alone can't express "contains, somewhere, at least one of each class" cleanly — so match the broad character class as one token and verify the three conditions with a scan inside the action.

14. Commands to run `assignment.1` against an input file given as a command-line argument. (5 marks)

```
flex assignment.l
g++ lex.yy.c -o scanner -lfl
./scanner input.txt
```

(`lex.yy.c` is Flex's generated scanner; `-lfl` links Flex's runtime library; the compiled binary reads `argv[1]` internally per the problem statement, so it's invoked with the filename as a plain argument, not via `<` redirection.)

15. How many conflicts does this grammar generate, and fix it? (5 marks)

```
P : D ;
D : ID COLON T {Two} | PROC ID {Three} | START S END {Four}
D : D SEMICOLON D {Five} | ;
T : INTEGER {Six} | REAL {Seven} ;
S : ID ASSIGNOP ID ;
S : S SEMICOLON S | ;
```

Model answer: The problematic pattern is `D : D SEMICOLON D | ;` (and identically `S : S SEMICOLON S | ;`) — a rule that both **self-concatenates on both sides** and allows an **empty** alternative. This is a classically ambiguous "list" construction: there's no unique way to decide, while parsing, whether a partially-seen `D` should reduce now (treating what follows as a **separate**, empty-tail `D`) or keep shifting to build a longer chain, and the empty alternative makes it worse by giving the parser an "it could already be done" option at almost every point. This produces multiple shift/reduce conflicts (and potentially reduce/reduce conflicts) — the exact count is best confirmed by actually compiling with `bison -v` and reading `y.output`, but expect on the order of a few (commonly cited as ~2 per such ambiguous self-referential list rule, so roughly 4 total across both `D` and `S`). **Fix — rewrite both as standard left-recursive lists with one base case, removing the empty self-referential alternative:**

```
P      : Dlist ;
Dlist  : Dlist SEMICOLON Decl | Decl ;
Decl   : ID COLON T {Two} | PROC ID {Three} | START Slist END {Four} ;
T      : INTEGER {Six} | REAL {Seven} ;
Slist  : Slist SEMICOLON Stmt | Stmt | /* empty */ ;
Stmt   : ID ASSIGNOP ID ;
```

This standard "list := list SEMI item | item" shape is unambiguous — there's exactly one way to build up the chain — and is the idiomatic YACC pattern for comma/semicolon-

separated lists in general (worth memorizing as a template, since "spot and fix the ambiguous list grammar" is a very reusable exam pattern).

16. Trace the corrected grammar on the given program and state the parser's output. (5 marks)

```
a: int;
b: real;
proc abc;
  x: int;
  y: real;
start
  x = y;
  a = b;
end
```

Model answer (method, since exact print *ordering* depends on how your specific parser resolves the (now-fixed) list grammar's shift order — walk through it explicitly in your answer rather than just stating a final sequence): Bottom-up parsing reduces the **innermost** rule matches first. So for **a: int;**, **T : INTEGER** reduces first (printing **Six**), then **Decl : ID COLON T** reduces (printing **Two**). Same shape for **b: real;** → **Seven**, then **Two**. **proc abc;** matches **Decl : PROC ID** directly → **Three**. Inside the **start...end** block: **x = y;** and **a = b;** both match **Stmt : ID ASSIGNOP ID** (no action attached, so nothing prints for these), combined via the **Slist** list rule (also no action), and the whole block reduces via **Decl : START Slist END** → **Four**. Finally, all the top-level declarations combine via the **Dlist** list rule (no action on the list rule itself) into one **Dlist**, and **P : Dlist** triggers... (*note: the original ungraded rule had an action {printf("One\n");} on P : D; keep that on P : Dlist in the fixed grammar*) → **One**, printed last, exactly once, since the whole program is a single top-level list. **Full print sequence: Six, Two, Seven, Two, Three, Six, Two, Seven, Two, Four, One.** (*If you used a different but equally-valid conflict fix than the one shown in Q15, your parser might interleave the list-combination differently — the important exam skill being tested is bottom-up reduction ordering, not memorizing this exact sequence.*)

17. Generate ICG for **statement : WHILE LPAREN expression RPAREN statement. (5 marks)** **Model answer**, in the **newLabel()/newTemp()** textbook style the question asks for, and also shown in the concrete x86 shape used throughout your course (see §5.4/§5.3 for why this shape looks the way it does):

```

L1 = newLabel(); L2 = newLabel();
emit(L1 + ":");
emit(expression.code);           // leaves condition result in
expression.addr
emit("CMP " + expression.addr + ", 0");
emit("JE " + L2);
emit(statement.code);
emit("JMP " + L1);
emit(L2 + ":");

```

This is the standard "test-at-top, unconditional-jump-back" loop shape: label the top of the loop, evaluate the condition, jump *out* past the loop body if it's false, otherwise fall through into the body, then unconditionally jump back to re-test.

18. Given a 3-column block of assembly, strike out dead lines and find the minimum number of temporaries. (5 marks) Model answer (method — see §5.5 for the full explanation): Read the block top-to-bottom, left-to-right across the three columns as the question specifies. For each temporary (**t0**, **t1**, **t2**, ...), mark its definition point and its last-use point. Any temporary whose defining store is never read before being overwritten (or before the block ends, given non-temporaries are live at block-end but temporaries are not) is a **dead store** — strike that line. Compute each surviving temporary's **live range** (definition → last use); the **minimum number of temporaries** needed equals the maximum number of these live ranges that are simultaneously open at any single instruction (the "overlap depth"). *This specific block's exact struck-out lines depend precisely on its column layout, which is easy to mis-transcribe from a scanned exam PDF — re-derive it directly against your own copy of the original quiz page using the method above rather than trusting a blindly copied answer.*

PART VIII — Fresh Practice Mock Quiz (new questions, same 50-mark shape, fully explained)

MCQ (1 mark each)

1. Which register holds the **remainder** after **IDIV**? — **EDX** (quotient is in EAX).

2. In ANTLR4, which pattern lets operator precedence be expressed directly in a single rule? — **Direct left recursion, with alternative order encoding precedence.**
3. Which command in the symbol-table "C1" style problem must search *every* open scope, not just the current one? — **DT <type> (type deletion).**
4. What does **NOT EAX** compute, as opposed to **NEG EAX**? — **Bitwise complement (flips every bit) vs. two's-complement arithmetic negation** — **NOT $x = -x - 1$** , they are *not* the same operation.
5. In the Mickey Lang interpreter problem, what should happen when an error (e.g., undeclared variable) is encountered? — **Skip only that line and continue; never terminate the program.**
6. Which ANTLR-generated file is the closest analogue to Bison's **y.tab.h**? — **The <Grammar>.tokens file.**
7. What is the defining difference between a "dead store" and an "unused variable"? — **A dead store concerns one specific write being clobbered/unread before the next write or scope exit; an unused variable concerns a name being read zero times across its entire lifetime.**
8. Why must an invalid numeric lexeme like **12UU** be consumed as a single erroneous token rather than split? — **The maximal-munch/longest-match principle — the scanner commits to the longest matching pattern as one lexeme, valid or not.**
9. What real assembler/linker problem is directly analogous to implementing forward-referencing **goto** labels? — **Resolving forward references, traditionally solved with a two-pass assembly process.**
10. Between a stack-based bytecode VM and a register-based target like x86, which one introduces the register-allocation problem? — **The register-based target (x86)** — a stack machine has no finite register file to run out of.

Short Questions (5 marks each, with explained model answers)

11. **Explain, with an example, why a variable declared via **IS** (scope-local) is visible to a lookup issued from its own scope but not to a lookup issued from an ancestor scope, even though ancestor scopes normally can't see child-scope variables anyway — what exactly is unusual here? Answer:**
Normally, a parent scope genuinely *cannot* see anything declared in a child scope (lookups only travel *outward*, from child to parent, never the reverse), so "invisible to ancestors" sounds automatic. The unusual case **IS** actually protects against is a

lookup issued **from within the scope-local symbol's own scope, on behalf of a deeper descendant scope** — normal symbols *are* visible to lookups from descendant scopes (that's the entire point of the parent-pointer walk), so without the special "scope-local" flag, a symbol declared with **IS** in scope 1 would still be found by an ordinary **L** issued from a nested scope 1.1, purely because 1.1's lookup walks up through scope 1. The flag makes **Lookup** explicitly skip such entries whenever the search has moved *outside* the exact scope that declared them — while still finding them instantly when the lookup originates in that same scope.

12. **Why does the "definitions pass, then references pass" two-pass pattern (from the ANTLR book's symbol table chapter) avoid a class of false-positive errors that a single combined pass would produce? Give a concrete example from your own CSubset language. Answer:** A single left-to-right pass that tries to define *and* check symbols simultaneously would flag any identifier used before its textual declaration point as "undeclared" — but your language explicitly allows forward references (e.g., a function called before its later declaration/definition, as long as it's declared/defined *somewhere* in the file). A definitions-first pass builds the *complete* picture of every declared symbol regardless of order; only then does a second pass check every *use* against that complete picture, so a legitimately forward-referenced function is correctly resolved instead of incorrectly flagged.
13. **A quiz gives you `int a; { int a; a = 5; } println(a);` — walk through exactly what the symbol table does at each line, and state what gets printed. Answer:** `int a;` declares **a** in the global scope (Scope 1). `{` enters a new scope (Scope 2, parent = Scope 1). `int a;` declares a *second*, distinct **a** inside Scope 2 — legal, because uniqueness is only checked *within* a scope, and this **a** shadows the outer one for the rest of Scope 2. `a = 5;` resolves to Scope 2's **a** (nearest enclosing declaration), setting it to 5. `}` exits Scope 2, discarding its **a** entirely. `println(a);` now resolves **a** in Scope 1 (Scope 2 is gone), which was **never assigned** in this snippet — so, depending on your language's rules about default-initialized globals, this prints **0** (the default value for an uninitialized global int), *not 5* — because the **a** that got **5** was a completely different variable that ceased to exist once its scope closed.
14. **Why is **CWD** required before **IDIV**, and what visible symptom would you see in your program's output if you forgot it? Answer:** **IDIV** divides the full 64-bit value held across the **EDX:EAX** register pair by its operand — it doesn't just look at **EAX** alone. **CWD** sign-extends whatever's currently in **EAX** into **EDX** (all 0s if **EAX** is

non-negative, all 1s if negative) so that **EDX:EAX** together represent the **same** value as **EAX** alone, just widened correctly. If you skip **CWD**, **EDX** retains whatever garbage was left over from an earlier computation, so the 64-bit dividend **IDIV** actually uses is wrong — the visible symptom is **wildly incorrect quotients/remainders** (or a divide-overflow crash) for otherwise-correct-looking arithmetic, and the bug would be intermittent-looking because it depends on what happened to be in **EDX** beforehand.

15. **Compare the marks distribution across your symbol-table extension problems (Merge: 3+3+2+2; Dead-store/unused: 4+6; Type-deletion/scope-local: 6+4) and explain what this tells you about which sub-feature is harder to implement correctly, independent of just "reading the numbers." Answer:** Marks in these assignments are typically weighted toward the parts of a feature that are easiest to get subtly wrong, not just the parts that take the most code. Dead-store detection (6 of 10 marks) requires tracking, for every variable, "was there a read strictly between this write and the next event" — a small stateful condition that's easy to get off-by-one on (e.g., forgetting that a read **exactly at** scope-exit still counts, or double-counting a dead store already reported). Type-deletion (6 of 10) requires traversing **every** open scope rather than just the current one, which is a structural change to how you iterate, not just an added **if**. In both cases, the higher-marked half is the part that requires genuinely different control flow from the "obvious" first attempt, which is exactly why it's worth more — a grader is rewarding correct handling of the *non-obvious* case, not just a working happy path.
16. **Why does your Assignment 3 spec explicitly forbid **switch-case** and consecutive relational/logical operators (**a < b < c**), while your ICG online problems *do* require handling chained bitwise operators (**1 | 2 & 4 | 8 * 3 & ~-256**)? Is this an inconsistency? Answer:** Not an inconsistency — it's a scoping decision about which parts of the language get the "hard" grammar-design treatment in which assignment. **a < b < c** is excluded because relational operators in C don't have a well-defined *chained* meaning the way math notation implies (**a<b<c** in C actually means (**a<b**)<c, comparing a boolean-as-int against **c**, which is rarely what's intended and would need its own explicit semantic ruling) — the spec sidesteps that ambiguity entirely by simply disallowing it. Bitwise operators, in contrast, chain perfectly naturally under ordinary left-associative binary-operator semantics (**a | b | c** unambiguously means (**a|b**)|c), so there's no ambiguity to sidestep, and testing a decently long chain (**1 | 2 & 4 | 8 * 3 & ~-256**) is instead a genuine, fair test of whether your grammar's precedence-and-

associativity declarations are correct across *multiple* operator tiers simultaneously (arithmetic, bitwise, and unary NOT all interacting in one expression).

17. **Explain why `println(a, b, x, y)` (multiple comma-separated expressions) is a comparatively "easy" grammar extension compared to something like `goto`.** **Answer:** Multiple-expression `println` only requires the grammar to accept a comma-separated *list* of expressions (a simple repetition, `expr (',' expr)*`) and the code-generation visitor to loop over that list, evaluating and printing each one independently, left to right, with no interaction between them and no forward-reference problem — each expression's code generation is exactly as self-contained as the single-expression case. `goto`, by contrast, introduces a genuine **non-local control-flow dependency**: the target label may not exist yet in the part of the tree you've currently visited, forcing either a pre-pass to collect all labels first or some other mechanism to patch up forward jumps — a fundamentally different (and harder) kind of problem than "do the same thing N times in a row."
18. **In the ANTLR4 book's bytecode-compiler chapter, why is a stack-based VM a "simpler first target" for teaching code generation, and what specific problem does your course's x86 target force you to solve that a stack machine would not?** **Answer:** On a stack machine, every sub-expression's result just gets pushed, and every operator just pops its operands and pushes its result — there is never a decision about *where* to put a value, because "on top of the stack" is always the answer. Targeting x86 forces you to actively decide **which register holds which value at each point**, and to explicitly save/restore values (via `PUSH/POP` to memory-backed stack slots) whenever you need more live values than you have registers for — this is the **register allocation** problem, and it's precisely why the "minimum number of temporaries" style optimization questions (§5.5) exist for your course's target but wouldn't even make sense to ask about a pure stack-machine target.

PART IX — Conceptual & "Thinking" Questions (design-level understanding, fully explained)

These go beyond "recall a fact" and test whether you understand *why* the course is built the way it is — exactly the kind of question a reference-book-based quiz tends to favor.

1. Why can ANTLR4 support direct left recursion when classical top-down (LL) parsers historically could not? A hand-written or table-driven top-down parser decides what to do next by looking at the *next unconsumed token* and picking a rule to expand — but for a left-recursive rule (`expr : expr '+' term | term`), expanding `expr` means immediately trying to match `expr` again, having consumed nothing, which is an infinite regress with no token ever advancing. ANTLR4 solves this not by parsing left-recursively at runtime, but by **transforming the grammar at generation time**: it detects the direct left-recursive pattern and rewrites it into the standard iterative/precedence-climbing form internally (conceptually similar to the manual `expr → term (('+' | '-') term)*` rewriting compiler-theory courses teach by hand), so you get to *author* the natural, precedence-by-alternative-order grammar while the generated parser never actually attempts left recursion itself.

2. Why does the assignment mandate the Visitor pattern instead of Listener or embedded grammar actions, specifically for a code-generating compiler (as opposed to, say, a simple syntax checker)? Code generation is inherently compositional and bottom-up: the assembly for `a + b` needs the assembly *and result location* for `a` and for `b` before it can be combined. The Visitor pattern's `visitX()` methods **return values**, so a parent rule's visitor method can call `visit()` on each child and directly receive back exactly the `{code, address}` information it needs to combine. The Listener pattern's `enterX/exitX` methods return nothing — you'd have to fake return values via an external map keyed by tree node, which is strictly more bookkeeping for no benefit here. Embedded actions are additionally banned because they'd scatter this logic across the grammar file itself, coupling syntax (what the language *looks* like) with semantics (what it *means*) — a design smell that makes both harder to change independently.

3. Why is a separate hash table *per scope* a better design than one global hash table where every entry additionally stores a "scope-id" field? With one hash table per scope, **exiting a scope is a single O(1) operation** — discard the whole table. With one shared global table and scope-tagged entries, exiting a scope would require **scanning the entire table** (or maintaining a separate auxiliary index of "which entries belong to scope N") just to find and remove that scope's entries — turning a constant-time operation into one proportional to total program size. The per-scope design also makes **shadowing trivial**: a name in an inner scope's own table simply exists alongside an unrelated same-named entry in an outer scope's table, with no collision-avoidance logic needed at all, whereas a single shared table would need every operation to additionally filter or prioritize by scope.

4. Why must dead-store detection consider "the next write, or scope exit" as the deadline for a read to occur, rather than simply "was this variable ever read, anywhere, for the rest of the program"? That broader question is a *different*, coarser property — "unused variable," which your course tracks separately. Dead-store analysis is deliberately more precise: it's about whether **this particular write's value ever mattered**, and a value stops mattering the moment it gets overwritten by the *next* write (nothing can ever read the old value again after that point, even if the variable itself goes on to be read many more times later) or the moment its scope ends (nothing can read it at all after that, from anywhere). Treating "ever read anywhere in the future" as the criterion would incorrectly call an assignment "live" even when a later, unrelated write had already made its specific value permanently unreachable — this distinction (per-write liveness vs. whole-variable usage) is precisely what real compiler liveness analysis computes, and it's why the two checks are genuinely separate, not two phrasings of the same thing.

5. Explain the maximal-munch principle and how it explains the recurring "don't split an invalid lexeme" rule across your lexical-analysis extensions. Maximal munch (longest match) says a scanner always consumes the *longest* possible prefix of the remaining input that matches *some* token pattern, before deciding what token that is. This is what makes scanning deterministic and unambiguous in the first place — without it, **12UU** could be sliced up many different ways (**1**,**2**,**U**,**U** as four tokens; **12**,**UU**; etc.) with no principled reason to prefer one slicing over another. Because the scanner is *already committed* to treating the whole run of digit-and-suffix characters as one attempted numeric-literal match (that's what longest-match did), discovering that the suffix is invalid doesn't retroactively un-commit that decision — the entire matched span is still "the one lexeme the scanner tried to interpret," so it's reported (and consumed) as a single erroneous unit, not silently re-split into a valid prefix plus leftover garbage tokens.

6. Why is $a = b = c = \text{expr}$ right-associative, and why does correct code generation evaluate expr only once rather than once per target variable? Right-associativity here means the grouping is $a = (b = (c = \text{expr}))$ — the *innermost* assignment (to **c**) happens "first" conceptually, and its *result* (the assigned value) is what gets assigned to **b**, and *that* result to **a**. Because C-family assignment is defined to *produce a value* equal to what was assigned, and expr is a single expression appearing textually once, the language semantics only call for evaluating it once — evaluating it three times (once "for" each target) would be observably different if expr had side effects (e.g., contained **d++**), incrementing **d** three times instead of once. Correct code generation reflects the *single* evaluation semantics: compute expr once into a register, then copy that one resulting value into **c**, **b**, and **a** in turn.

7. Why does `IDIV` need `CWD` first, and what does this reveal about the general principle that "an instruction's calling convention is part of its specification, not a stylistic choice"? As explained in Part V, `IDIV` always operates on the 64-bit value spread across `EDX:EAX`, by hardware design — it's not optional or configurable. This illustrates a broader compiler-construction lesson: code generation isn't just "translate this operator to that instruction," it's translating to instructions **plus whatever setup those specific instructions require to be well-defined** (sign-extension before divide, clearing/setting flags before certain conditional instructions, etc.). A code generator that doesn't model these calling-convention-like requirements per instruction will produce code that *assembles* fine (FASM won't complain) but computes wrong answers at runtime — exactly the class of bug that's hardest to debug because there's no compiler error to point you at it.

8. Compare stack-based (0-address) intermediate code with register-based (3-address-style, x86) intermediate code — what does each optimize for, and why does your course use the harder one? A stack machine's code generator never has to decide *where* a value lives — everything is "on top of the stack," so generation is close to a direct, mechanical translation of the expression tree's structure, with no separate register-allocation problem to solve. A register machine (your x86 target) requires the generator to actively track which register holds which live value, spill values to memory (via `PUSH`/stack slots) when there aren't enough registers, and reason about the *lifetime* of each value — genuinely harder, but also **far closer to how real native compilers actually work**, since real CPUs have a small, fixed set of fast registers and no infinite implicit stack of "current values." Your course targets the harder, more realistic model specifically because that's the skill (and the accompanying optimization problems, like minimizing temporaries) that transfers to real-world compiler and low-level systems work.

9. Why does implementing forward-referencing `goto` essentially force a two-pass (or pre-scan) design, and how does this generalize beyond just `goto`? A single left-to-right code-generation pass can only emit a jump to a label it has *already assigned an address to* — if the label appears later in the source, the generator doesn't yet know what to call it (or, in a real assembler, what numeric address it will end up at). The standard fix is to either (a) pre-scan the whole tree once, purely to register every label's identity before generating any jump instructions referencing it, or (b) emit a placeholder and patch it in a second pass once everything is known. This is the exact same shape of problem as: resolving mutually-recursive function calls, resolving a variable used before its declaration when forward references are allowed (§4A's two-pass symbol table pattern), and how real linkers resolve symbols across multiple compiled object files — "some references can only be resolved once you have global knowledge of the whole

program, not just what's been seen so far" is a recurring compiler/systems theme, not a one-off `goto` quirk.

10. Why is STL disallowed in the symbol-table assignment but explicitly permitted in the ICG assignment? The symbol-table assignment's entire *pedagogical point* is to make you build and manage the hash table, chaining, and scope-stack data structures yourself — using `std::unordered_map` or `std::vector` would let you skip the exact mechanics (hashing, chaining, manual memory ownership across scope enter/exit) that are the actual learning objective, so it's banned to force you to engage with them directly (and, not coincidentally, this is also why `-fsanitize=address` matters here — manual memory management is exactly where leaks/use-after-free bugs come from). The ICG assignment's learning objective is different — it's about correctly modeling *code generation* and x86 conventions, not about re-proving you can implement a hash map, so ordinary C++ housekeeping (e.g., using a `std::map` to track label names or function signatures) is allowed because it isn't the thing being tested there.

11. Why can't semantic checks like "undeclared variable" or "type mismatch" be expressed as part of the context-free grammar itself, requiring a separate semantic-analysis phase instead? Context-free grammars describe **shape** — legal arrangements of tokens — using a fixed, finite set of rules that have no memory of *which specific identifiers* were declared earlier in this particular program, or what type each one has. "Is `x` declared?" and "does `x`'s type match this context?" are properties of *this specific program's history* (which names appeared in which declarations), not properties of token arrangement in general — and grammars can't parameterize their rules by an unbounded, program-specific set of names. This is precisely why these checks need a symbol table (a piece of mutable, program-specific state built up as you go) and a separate analysis pass that consults it, rather than being foldable into the grammar's fixed set of production rules.

12. Why is `-fsanitize=address` specifically apt for these assignments' style of C++ (raw `new/delete`, manual pointer-based chaining and scope-table teardown), rather than being generic advice for any C++ project? Any project using RAII/smart pointers/STL containers gets most of ASan's protections "for free" simply by construction — there's rarely a raw `delete` to forget or a raw pointer to dangle. Your symbol-table implementation, by contrast, is *required* to manage memory by hand (`new SymbolInfo(...)`, explicit chaining via raw `next` pointers, explicit `delete` in `ScopeTable`'s destructor, explicit scope-stack push/pop tied to explicit object lifetime) — which is exactly the situation where it's easy to forget a `delete` (a leak) or to keep using a `ScopeTable*` after its scope has already been popped and destroyed (a use-

after-free, e.g., a stale `parent_scope` pointer). ASan instruments memory accesses and allocator calls at compile time specifically to catch these two bug classes the instant they happen, with a clear crash-and-report instead of silent corruption that might only surface as a mysterious wrong answer several operations later — which is why it's not just generic hygiene advice here, it's a targeted response to the exact bug profile this assignment's required coding style produces.

Final Study-Order Recommendation

1. Part V (ICG/x86) — heaviest represented topic, drill the worked trace in §5.3 until you can reproduce it cold for a new snippet.
2. Part III (Lexical) — internalize the maximal-munch principle (§3.6) since it explains half the "gotchas."
3. Part II (Symbol Table) — the four case studies (§2.5) are your highest-yield "spot the new twist" practice.
4. Part IV + Part 4A/4B (ANTLR4 theory) — this is explicitly where your teacher said questions come from; know the Listener-vs-Visitor reasoning and the two-pass symbol-table pattern by heart.
5. Part VII (old quiz, fully explained) and Part VIII (fresh mock) — run these cold, then re-read only what you missed.
6. Part IX (conceptual/thinking questions) — read these last, since they assume you already know Parts I–V; they're where the real 5-mark "explain why" answers live.